

✦ We have now addressed the major caveats of turning the simulation into a professional, production-ready project.

To recap:

1. **Environment:** We have a sandboxed Python environment with explicit dependencies.

2. **Containerization:** The application is fully containerized with Docker and can be orchestrated with Docker Compose.

3. **Testing:** We have an automated testing framework (pytest) with initial unit tests.

4. **CI/CD:** We have a GitHub Actions workflow for continuous integration.

5. **Documentation:** We have a README.md that explains how to set up, run, and test the project.

The project is now in a state where a team of engineers could effectively collaborate on it, and it is structured for deployment

into a real-world cloud environment using standard DevOps practices. The next step would be to replace the file-based data

simulations with actual managed services (e.g., a real Kafka cluster and an S3 bucket), which would involve adding credentials

and client libraries, for which I have already left placeholders.

The production-grade setup is complete.

✦ Of course. This is the final and most critical phase: transitioning our production-ready simulation into a true, cloud-native distributed system. I will provide a comprehensive, step-by-step guide to achieve

this.

This guide will use **Amazon Web Services (AWS)** as the cloud provider and **Kubernetes** for orchestration, which is a standard, powerful combination for modern applications. I will provide the necessary configuration files and code modifications, clearly marking all placeholders for credentials and environment-specific values.

Phase 1: Provisioning Cloud Infrastructure with Terraform

First, we need to create our cloud infrastructure. We will use Terraform to define our infrastructure as code (IaC), which makes the setup automated, repeatable, and version-controlled.

Action: Create a new directory named `terraform/` and place the following files inside it.

1. ``terraform/variables.tf``

This file defines the variables we'll use.

```
1 variable "aws_region" {
2   description = "The AWS region to deploy
resources in."
3   type       = string
4   default    = "us-east-1"
5 }
6
7 variable "project_name" {
8   description = "A unique name for the
project to prefix resources."
9   type       = string
```

```

10     default      = "leaderboard"
11 }
12
13 variable "vpc_id" {
14     description = "The ID of the VPC to
deploy into. Replace with your default or a
specific VPC."
15     type        = string
16     # PLACEHOLDER: Replace with your actual
VPC ID
17     default      = "vpc-0123456789abcdef0"
18 }
19
20 variable "subnet_ids" {
21     description = "A list of subnet IDs for
Kafka and Kubernetes."
22     type        = list(string)
23     # PLACEHOLDER: Replace with your actual
subnet IDs
24     default      = ["subnet-012345abc",
"subnet-678901def"]
25 }

```

2. `terraform/main.tf`

This file defines the actual resources: an S3 bucket, a Kafka cluster (MSK), and a Kubernetes cluster (EKS).

```

1 terraform {
2     required_providers {
3         aws = {
4             source  = "hashicorp/aws"
5             version = "~> 5.0"
6         }
7     }
8 }
9
10 provider "aws" {
11     region = var.aws_region

```

```

12 }
13
14 # 1. S3 Bucket to replace HDFS/local file
storage
15 resource "aws_s3_bucket" "data_lake" {
16     bucket = "${var.project_name}-data-lake-${
{random_id.bucket_suffix.hex}"
17 }
18
19 resource "random_id" "bucket_suffix" {
20     byte_length = 8
21 }
22
23 # 2. Managed Kafka (MSK) Cluster
24 resource "aws_msk_cluster" "kafka_cluster"
{
25     cluster_name          = "$
{var.project_name}-kafka-cluster"
26     kafka_version          = "3.5.1"
27     number_of_broker_nodes = 2
28
29     broker_node_group_info {
30         instance_type = "kafka.m5.large"
31         client_subnets = var.subnet_ids
32         storage_info {
33             ebs_storage_info {
34                 volume_size = 100
35             }
36         }
37     }
38 }
39
40 # 3. Kubernetes (EKS) Cluster
41 resource "aws_eks_cluster" "eks_cluster" {
42     name          = "${var.project_name}-eks-
cluster"
43     role_arn =
aws_iam_role.eks_cluster_role.arn # Assumes an
IAM role is defined elsewhere

```

```
44
45     vpc_config {
46         subnet_ids = var.subnet_ids
47     }
48 }
49
50 # NOTE: A production Terraform setup would
also define IAM roles,
51 # security groups, and EKS node groups.
These are simplified for clarity.
52 # You would need to create
aws_iam_role.eks_cluster_role separately.
```

How to use this:

1. Install Terraform (<https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli>).
2. Configure your AWS credentials locally.
3. Fill in the PLACEHOLDER values in variables.tf.
4. Run the following commands from within the terraform/ directory:
 - 1 terraform init
 - 2 terraform plan
 - 3 terraform apply

Phase 2: Modifying the Application Code

Now, we must update our Python services to communicate with the newly provisioned S3 and Kafka instead of the local filesystem.

Action: Update requirements.txt and the service code.

1. Update `requirements.txt`

Add the necessary AWS and Kafka client

libraries.

```
✓ WriteFile Writing to requirements.txt
```

```
1  ruff
```

```
2  pytest
```

```
3 + boto3
```

```
4 + kafka-python
```

```
5 + prometheus-client
```

✦ 2. Update `realtime\_processor.py`

Replace file operations with S3 and Kafka clients.

* **Configuration:** At the top of the file, we'll now get configuration from environment variables.

```
1      # --- Configuration ---
2      S3_BUCKET_NAME =
os.environ.get('S3_BUCKET_NAME')
3      KAFKA_BROKERS =
os.environ.get('KAFKA_BROKERS').split(',')
4      KAFKA_TOPIC =
os.environ.get('KAFKA_TOPIC',
'leaderboard_events')
```

```
5
6     s3_client = boto3.client('s3')
7     SKETCH_STATE_KEY = 'state/
cms_state.json'
    * Loading the Sketch: Replace load_sketch to
fetch from S3.
```

```
1     def load_sketch():
2         sketch =
CountMinSketch(width=CMS_WIDTH, depth=CMS_DEPTH)
3         try:
4             response =
s3_client.get_object(Bucket=S3_BUCKET_NAME,
Key=SKETCH_STATE_KEY)
5             state =
json.loads(response['Body'].read().decode('utf-8'
))
6             sketch.sketch = state
7             print("Loaded existing Count-
Min Sketch from S3.")
8         except
s3_client.exceptions.NoSuchKey:
9             print("Created a new Count-Min
Sketch.")
10            return sketch
    * Saving the Sketch: Replace save_sketch to
write to S3.
```

```
1     def save_sketch(sketch):
2         s3_client.put_object(
3             Bucket=S3_BUCKET_NAME,
4             Key=SKETCH_STATE_KEY,
5             Body=json.dumps(sketch.sketch)
6         )
7         print(f"Saved Count-Min Sketch state
to S3 bucket {S3_BUCKET_NAME}.")
    * Processing Events: Replace the main loop to
consume from Kafka.
```

```

1      def process_events():
2          sketch = load_sketch()
3          consumer = KafkaConsumer(
4              KAFKA_TOPIC,
5              bootstrap_servers=KAFKA_BROKERS,
6              auto_offset_reset='earliest',
7              group_id='realtime-processor-
group',
8              value_deserializer=lambda v:
json.loads(v.decode('utf-8'))
9          )
10         print("Starting Kafka consumer for
real-time processing...")
11         for message in consumer:
12             event = message.value
13             sketch.add(event['event_id'])
14             # In a real system, you'd save
periodically, not on every message
15             save_sketch(sketch)

```

3. Update `batch\_processor.py`

Similarly, update this processor to use Kafka and S3.

* **Main Logic:** It will consume from Kafka (with a different group_id) and write its batch output to S3 instead of the local HDFS_DIR.

```

1      # ... (imports and configuration
similar to realtime_processor)
2
3      def process_batch():
4          # This would be a scheduled Spark/
Flink job in a real system.
5          # This simulation consumes a number
of messages and then writes a batch.
6          consumer = KafkaConsumer(

```



```

7             KAFKA_TOPIC,
8
bootstrap_servers=KAFKA_BROKERS,
9             auto_offset_reset='earliest',
10            group_id='batch-processor-
group',
11            consumer_timeout_ms=10000, #
Stop if no messages for 10s
12            value_deserializer=lambda v:
json.loads(v.decode('utf-8'))
13        )
14
15        batch_counts = Counter()
16        message_count = 0
17        for message in consumer:
18            event = message.value
19            batch_counts[event['event_id']]
+= 1
20            message_count += 1
21
22            if message_count > 0:
23                output_data = [{'event_id':
eid, 'count': c} for eid, c in
batch_counts.items()]
24                batch_timestamp =
int(time.time())
25                output_key = f'batches/
batch_{batch_timestamp}.json'
26
27                s3_client.put_object(
28                    Bucket=S3_BUCKET_NAME,
29                    Key=output_key,
30
Body=json.dumps(output_data)
31                )
32                print(f"Saved batch file to
s3://{S3_BUCKET_NAME}/{output_key}")

```

Phase 3: Kubernetes Deployment

With the application code updated, we can now define how to run it on our EKS cluster.

Action: Create a `k8s/` directory and add the following YAML files.

1. ``k8s/configmap.yaml``

This file externalizes our configuration, so we don't have to hardcode it in the Docker image.

```
1 apiVersion: v1
2 kind: ConfigMap
3 metadata:
4   name: leaderboard-config
5 data:
6   # PLACEHOLDER: Replace with your actual
S3 bucket name from Terraform output
7   S3_BUCKET_NAME: "leaderboard-data-
lake-..."
8   # PLACEHOLDER: Replace with your actual
Kafka brokers from Terraform output
9   KAFKA_BROKERS: "b-1.kafka.us-
east-1.amazonaws.com:9092,b-2.kafka.us-
east-1.amazonaws.com:9092"
10  KAFKA_TOPIC: "leaderboard_events"
```

2. ``k8s/deployment.yaml``

This file describes how to run our services as scalable, resilient deployments on Kubernetes.

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: realtime-processor
5 spec:
6   replicas: 2 # Run two instances for high
```

```
availability
  7   selector:
  8     matchLabels:
  9       app: realtime-processor
 10   template:
 11     metadata:
 12       labels:
 13         app: realtime-processor
 14     spec:
 15       containers:
 16       - name: processor
 17         # PLACEHOLDER: Replace with your
 18         Docker image registry URL
 19         image: "123456789012.dkr.ecr.us-
 20         east-1.amazonaws.com/leaderboard:latest"
 21         command: ["python3", "src/
 22         processors/realtime_processor.py"]
 23         envFrom:
 24         - configMapRef:
 25           name: leaderboard-config
 26 ---
 27 apiVersion: apps/v1
 28 kind: Deployment
 29 metadata:
 30   name: leaderboard-api
 31 spec:
 32   replicas: 2
 33   selector:
 34     matchLabels:
 35       app: leaderboard-api
 36   template:
 37     metadata:
 38       labels:
 39         app: leaderboard-api
 40     spec:
 41       containers:
 42       - name: api
 43         # PLACEHOLDER: Replace with your
 44         Docker image registry URL
```

```

41         image: "123456789012.dkr.ecr.us-
east-1.amazonaws.com/leaderboard:latest"
42         command: ["python3", "src/
leaderboard_service/service.py"] # This should be
a real API server
43         envFrom:
44         - configMapRef:
45             name: leaderboard-config
46 ---
47 # A CronJob is a better fit for the batch
processor
48 apiVersion: batch/v1
49 kind: CronJob
50 metadata:
51   name: batch-processor
52 spec:
53   schedule: "0 * * * *" # Run every hour
54   jobTemplate:
55     spec:
56       template:
57         spec:
58           containers:
59           - name: batch-processor
60             # PLACEHOLDER: Replace with
your Docker image registry URL
61             image:
"123456789012.dkr.ecr.us-east-1.amazonaws.com/
leaderboard:latest"
62             command: ["python3", "src/
processors/batch_processor.py"]
63             envFrom:
64             - configMapRef:
65                 name: leaderboard-config
66             restartPolicy: OnFailure

```

How to handle credentials:

Your EKS pods will need permission to access S3 and MSK. The best practice is to use **IAM Roles for Service Accounts (IRSA)**. You

would create an IAM role with the necessary permissions, and then associate it with the Kubernetes service account that the pods run under. This avoids ever having to handle raw AWS keys in your application or configuration.

Phase 4: Monitoring with Prometheus

Action: Install Prometheus and configure it to scrape metrics from our application.

1. Instrument the Application

Modify the Python code (e.g., in the `leaderboard-api`) to expose a `/metrics` endpoint.

```
1 from prometheus_client import
start_http_server, Counter
2
3 # At the top of your service
4 REQUEST_COUNTER =
Counter('api_requests_total', 'Total number of
API requests')
5
6 # In your request handling logic
7 REQUEST_COUNTER.inc()
8
9 # At the end of your main block
10 if __name__ == '__main__':
11     start_http_server(8000) # Expose
metrics on port 8000
12     # ... rest of your service startup
```

2. Install Prometheus

Use the official Helm chart to install Prometheus and Alertmanager into your K8s cluster.

```
1 helm repo add prometheus-community https://
prometheus-community.github.io/helm-charts
2 helm install prometheus prometheus-
community/kube-prometheus-stack
```

3. Configure Scraping

You would configure the Prometheus Operator to discover and scrape pods with a specific annotation, for example

prometheus.io/scrape: 'true', which you would add to your deployment.yaml.

4. Configure Alerting

In the Helm chart's values, you would define alerting rules.

* **`alert.rules.yml` (example):**

```
1      groups:
2      - name: leaderboard-alerts
3        rules:
4        - alert: HighApiErrorRate
5          expr: rate(api_errors_total[5m]) >
0.1
6          for: 1m
7          labels:
8            severity: critical
9          annotations:
10             summary: "High API error rate on
leaderboard service!"
```

* **Alertmanager Configuration:** You would also configure Alertmanager to send notifications to Slack, PagerDuty, etc. This involves setting a webhook URL.

```
1      # In the helm values file for
alertmanager
2      config:
```

```
3      global:
4          resolve_timeout: 5m
5      receivers:
6      - name: 'slack-notifications'
7          slack_configs:
8      - api_url: 'https://hooks.slack.com/services/T000000000/B000000000/
XXXXXXXXXXXXXXXXXXXXXXXXXXXXX' # PLACEHOLDER
9          channel: '#alerts'
10     route:
11     receiver: 'slack-notifications'
```

This guide provides the templates and step-by-step methodology to deploy the system in a real-world setting. Each PLACEHOLDER indicates where you must insert your own specific resource names, account IDs, and credentials.